

SC2005 Operating Systems
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-23 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

1	Introduction to Operating Systems	1
1.1	What Is an Operating System?	1
1.1.1	Computer-System Organization	2
1.1.2	Storage Hierarchy	2
1.2	Dual-Mode Operation and Protection	2
1.3	System Calls and OS Services	2
1.4	OS Structures and Booting	3
1.5	Exercises	4
2	Processes and Threads	5
2.1	The Process Abstraction	5
2.1.1	Process States	5
2.1.2	Context Switch	5
2.2	Process Creation: <code>fork</code> and <code>exec</code>	6
2.3	Threads	6
2.3.1	User vs. Kernel Threads	7
2.3.2	Pthreads Example	7
2.3.3	IPC	7
2.4	Exercises	8
3	CPU Scheduling	9
3.1	Scheduling Concepts	9
3.1.1	Criteria	9
3.2	Classical Algorithms	10

3.2.1	FCFS (First-Come, First-Served)	10
3.2.2	SJF / SRTF	10
3.2.3	Priority Scheduling	10
3.2.4	Round Robin (RR)	10
3.2.5	Choosing the Quantum	11
3.3	Multilevel Feedback Queue (MLFQ)	11
3.3.1	Multicore and Real-Time	11
3.4	Exercises	12
4	Synchronization	13
4.1	Races and the Critical Section	13
4.1.1	Critical-Section Requirements	13
4.1.2	Peterson's Algorithm (2 processes)	14
4.1.3	Hardware Support	14
4.2	Semaphores and Mutexes	14
4.2.1	Producer–Consumer with Semaphores	14
4.3	Monitors and Condition Variables	15
4.3.1	The Readers–Writers Problem	16
4.4	Exercises	16
5	Deadlocks	17
5.1	System Model and Four Conditions	17
5.2	Resource-Allocation Graphs	18
5.3	Prevention and Avoidance	18
5.3.1	Banker's Algorithm	18
5.3.2	Dining Philosophers	19
5.4	Exercises	19
6	Memory Management	21
6.1	Address Binding and Spaces	21
6.1.1	Contiguous Allocation	21

6.2	Paging	22
6.2.1	TLB	22
6.2.2	Multi-Level Page Tables	22
6.3	Segmentation and Combined Schemes	23
6.4	Exercises	23
7	Virtual Memory	25
7.1	Demand Paging	25
7.1.1	Copy-on-Write and Memory-Mapped Files	26
7.2	Page Replacement	26
7.3	Thrashing and Frame Allocation	27
7.3.1	Frame Allocation and Locality	27
7.4	Exercises	27
8	File Systems, I/O, and Protection	29
8.1	Files and Allocation	29
8.1.1	Directories and Journaling	30
8.2	Disk Scheduling and I/O	30
8.2.1	I/O Subsystem	31
8.3	Protection and Security	31
8.4	Exercises	31

Chapter 1

Introduction to Operating Systems

“The operating system is the most important program that runs on a computer.” This chapter answers: what an OS *is*, why hardware needs it, and how it mediates every interaction between user programs and the machine.

Learning Outcomes. After this chapter you will be able to:

- (L1) explain the OS as resource manager and extended machine;
- (L2) distinguish kernel vs. user mode and the role of dual-mode operation;
- (L3) describe system calls, traps, and the system-call interface;
- (L4) sketch major OS structures (monolithic, layered, microkernel, modular);
- (L5) outline booting, interrupts, and I/O organization.

1.1 What Is an Operating System?

An operating system (OS) is system software that (i) manages hardware resources (CPU, memory, storage, I/O) and (ii) provides a convenient execution environment for applications. Two complementary views:

- **Resource allocator:** arbitrates conflicting requests for efficient, fair use; decides between users/jobs when resources are scarce.
- **Control program:** prevents errors and improper use of the computer.
- **Extended / virtual machine:** hides hardware complexity behind clean abstractions (files, processes, sockets).

Definition 1.1 (Kernel). The *kernel* is the core privileged component that runs with full hardware access, handling scheduling, memory management, file systems, and device drivers. Everything outside the kernel runs in *user mode* with restricted privileges.

1.1.1 Computer-System Organization

A modern system has CPUs, memory, and I/O devices connected via a bus. Device controllers buffer data and interrupt the CPU when work completes. Direct Memory Access (DMA) lets controllers transfer blocks without per-byte CPU involvement.

1.1.2 Storage Hierarchy

Registers → cache → RAM → SSD/HDD → network storage: speed decreases while capacity and cost-per-bit improve. The OS migrates data across this hierarchy transparently (caching, paging, buffering).

1.2 Dual-Mode Operation and Protection

To protect the OS from user programs (and users from each other), hardware provides two modes indicated by a mode bit:

- **Kernel (privileged) mode:** all instructions allowed.
- **User mode:** privileged instructions (I/O, halt, set mode bit) trap to the kernel.

A *privileged instruction* executed in user mode causes a trap (exception). Mode switches occur on system calls, interrupts, and exceptions. Timers prevent infinite loops from seizing the CPU.

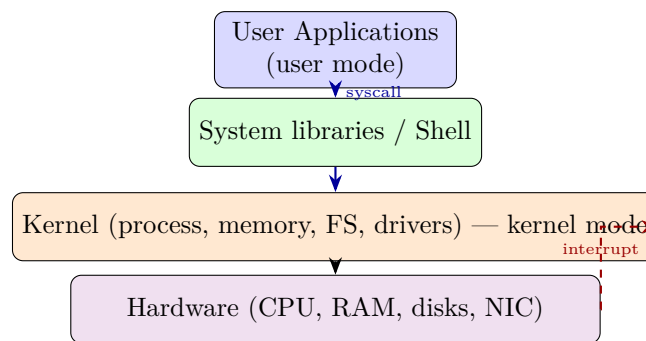


Figure 1.1: Layered view: user vs. kernel, hardware, and control flow.

1.3 System Calls and OS Services

A *system call* is the programmatic interface to OS services. It triggers a synchronous trap (e.g., `syscall` / `int 0x80` / `SVC`) that switches to kernel mode, dispatches via a system-call table, and returns.

Listing 1.1: System-call usage: copy a file (POSIX)

```

1 #include <fcntl.h>
2 #include <unistd.h>
3 #define BS 4096
4 int main() {

```

Category	Examples (POSIX)
Process control	fork, exec, exit, wait
File management	open, read, write, close
Device management	ioctl, read, write
Information	getpid, alarm, sleep
Communication	pipe, shmget, mmap, socket
Protection	chmod, umask, chown

Table 1.1: System-call categories with POSIX examples.

```

5  int src = open("in.txt", O_RDONLY);
6  int dst = open("out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644);
7  char buf[BS]; ssize_t n;
8  while ((n = read(src, buf, BS)) > 0) // read() is a system call
9      write(dst, buf, n);             // write() traps to kernel
10 close(src); close(dst);
11 return 0;
12 }

```

Types of system calls include process control, file manipulation, device management, information maintenance, communications, and protection.

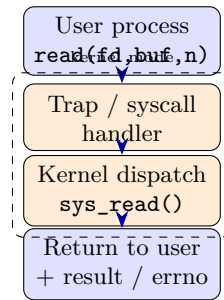


Figure 1.2: System-call path: trap, dispatch, and return.

1.4 OS Structures and Booting

Monolithic kernels (Linux, classic UNIX) run all services in one privileged image for performance. **Layered** designs enforce hierarchy; **microkernels** (Mach, seL4) keep only scheduling, IPC, and MM in the kernel, pushing drivers/FS to user servers. **Modular/hybrid** kernels load modules dynamically.

Booting: firmware (BIOS/UEFI) → bootloader → kernel image → `init/systemd` (PID 1). Interrupts (hardware) and exceptions (software faults) are vectored through an interrupt descriptor table.

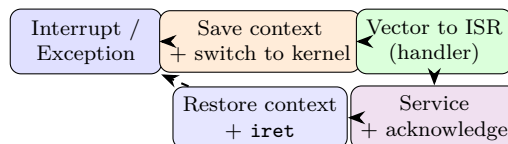


Figure 1.3: Interrupt handling pipeline (simplified).

Remark 1.1 (System-call cost). A system call is orders of magnitude slower than a function call due to mode switch, parameter validation, and cache/TLB effects. Hence `read` with a 1-byte buffer is far slower than buffering 4 KB and issuing fewer calls.

1.5 Exercises

Exercise 1.1. Explain why dual-mode operation is necessary. What goes wrong if all code runs in kernel mode?

Exercise 1.2. List three abstractions an OS provides over raw hardware and justify each.

Exercise 1.3. Trace the system calls issued when a shell runs `cp a.txt b.txt`. Which calls handle process creation vs. file I/O?

Exercise 1.4. Compare monolithic, microkernel, and modular structures on performance, reliability, and extensibility.

Exercise 1.5. Distinguish interrupt, exception, and system call (trap) along dimensions: synchrony, source, and handler lookup.

Chapter 2

Processes and Threads

A process is a program in execution. Understanding how the OS creates, schedules, and isolates processes — and how threads share within a process — is the foundation for concurrency.

Learning Outcomes. After this chapter you will be able to:

- (L1) define process, PCB, and process states; explain context switching;
- (L2) use `fork`, `exec`, `wait`, and `exit`;
- (L3) distinguish threads vs. processes and user vs. kernel threads;
- (L4) reason about multithreading models and pthreads programming;
- (L5) describe IPC mechanisms (pipes, shared memory, message passing).

2.1 The Process Abstraction

A *program* is passive (code on disk); a *process* is active, with a program counter, registers, address space (text/data/heap/stack), and OS resources (open files, signals). Each process has a *Process Control Block* (PCB) storing PID, state, PC, registers, scheduling info, memory limits, open files, and accounting.

2.1.1 Process States

New $\rightarrow$ *Ready* $\rightarrow$ *Running* $\leftrightarrow$ *Waiting/Blocked* $\rightarrow$ *Terminated*. The scheduler moves *Ready* $\rightarrow$ *Running* (dispatch); I/O or `wait` moves *Running* $\rightarrow$ *Waiting*; interrupt/completion moves *Waiting* $\rightarrow$ *Ready*.

2.1.2 Context Switch

On a timer interrupt or blocking call, the kernel saves the running process's registers/PC into its PCB, selects the next *Ready* process, restores its context, and resumes. This is pure overhead — no user work is done — so fast switching matters.

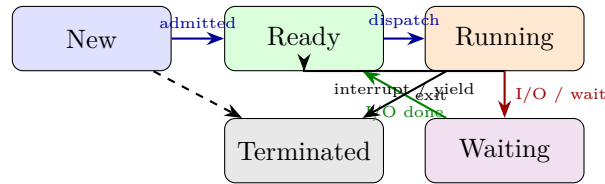


Figure 2.1: Five-state process model and transitions.

2.2 Process Creation: fork and exec

In UNIX, `fork()` creates a child that is a near-clone of the parent (copy of address space with copy-on-write, duplicated file descriptors). It returns 0 in the child and the child's PID (> 0) in the parent (or -1 on failure). `exec*()` replaces the current image with a new program.

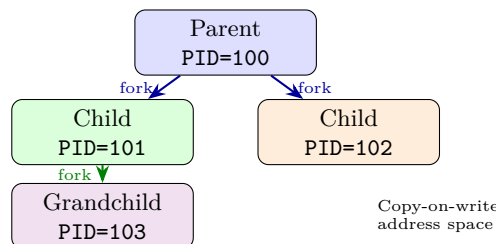
Listing 2.1: fork-exec-wait pattern (POSIX)

```

1 #include <unistd.h>
2 #include <sys/wait.h>
3 #include <stdio.h>
4 int main() {
5     pid_t pid = fork();
6     if (pid == 0) {
7         // child
8         execlp("ls", "ls", "-l", NULL); // replaces child image
9         perror("exec"); _exit(1);
10    } else if (pid > 0) {
11        int status; waitpid(pid, &status, 0); // parent waits
12        printf("child exited: %d\n", WEXITSTATUS(status));
13    } else { perror("fork"); }
14    return 0;
15 }

```

`wait/waitpid` lets a parent collect a child's exit status; `exit(status)` terminates a process. An orphan is reaped by `init`; a zombie retains its PCB until the parent waits.

Figure 2.2: Process tree formed by successive `fork` calls.

2.3 Threads

A *thread* is a schedulable unit within a process: it has its own PC, registers, and stack, but shares text, data, heap, and open files with peer threads. Benefits: responsiveness, resource sharing, economy (cheaper to create than a process), and parallelism on multicore.

2.3.1 User vs. Kernel Threads

User threads are managed by a library (fast switch, but a blocking syscall blocks the whole process). *Kernel threads* are OS-managed (heavier, but independently schedulable). Hybrid *many-to-many* maps m user threads onto n kernel threads.

2.3.2 Pthreads Example

Listing 2.2: POSIX threads: parallel sum

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #define N 1000000
4  long sum = 0; pthread_mutex_t lk = PTHREAD_MUTEX_INITIALIZER;
5  void *worker(void *arg) {
6      long lo = ((long*)arg)[0], hi = ((long*)arg)[1];
7      long local = 0;
8      for (long i = lo; i < hi; i++) local += i;
9      pthread_mutex_lock(&lk); sum += local; pthread_mutex_unlock(&lk);
10     return NULL;
11 }
12 int main() {
13     pthread_t t1, t2; long r1[]={0,N/2}, r2[]={N/2,N};
14     pthread_create(&t1,NULL,worker,r1);
15     pthread_create(&t2,NULL,worker,r2);
16     pthread_join(t1,NULL); pthread_join(t2,NULL);
17     printf("sum=%ld\n", sum);
18     return 0;
19 }
```

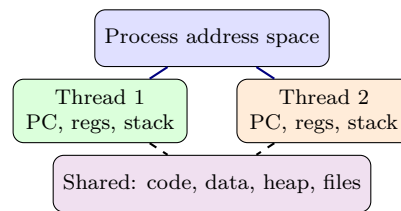


Figure 2.3: Single process, two threads sharing address space.

2.3.3 IPC

Cooperating processes need IPC: **pipes** (pipe), **shared memory** (shmget/mmap + synchronization), and **message passing** (msgsnd/msgrcv, sockets).

Remark 2.1 (Copy-on-write). After **fork**, parent and child share physical pages read-only. Only on a write does the kernel copy the page, making **fork** cheap even for large address spaces. **exec** then discards the copied mapping entirely.

2.4 Exercises

Exercise 2.1. Draw the PCB contents. Which fields must be saved on a context switch and why?

Exercise 2.2. What does the following print? Explain copy-on-write.

```
1 int x=10; if (fork()==0) { x++; printf("child %d\n",x); }  
2 else { wait(NULL); printf("parent %d\n",x); }
```

Exercise 2.3. Compare processes vs. threads on isolation, creation cost, and communication. When would you choose each?

Exercise 2.4. Explain many-to-one, one-to-one, and many-to-many thread models and their trade-offs for blocking I/O and parallelism.

Exercise 2.5. Write a program where parent and child communicate via a pipe: child sends a message, parent prints it.

Chapter 3

CPU Scheduling

Scheduling decides which Ready process runs next. A good scheduler balances fairness, throughput, and responsiveness under conflicting goals.

Learning Outcomes. After this chapter you will be able to:

- (L1) define CPU vs. I/O bursts and scheduling criteria;
- (L2) compute waiting/turnaround/response times for FCFS, SJF, SRTF, RR, Priority;
- (L3) explain MLFQ design and its approximation of SJF without prior knowledge;
- (L4) reason about multicore, real-time, and Linux CFS scheduling;
- (L5) evaluate schedulers via Gantt charts and queueing analysis.

3.1 Scheduling Concepts

Each process alternates CPU bursts and I/O bursts. The scheduler selects among Ready processes when the CPU becomes idle or a quantum expires. Scheduling is *nonpreemptive* (process keeps CPU until it blocks/exits) or *preemptive* (OS can interrupt).

3.1.1 Criteria

- **CPU utilization:** keep the CPU busy.
- **Throughput:** jobs completed per unit time.
- **Turnaround time:** submission $\rightarrow$ completion.
- **Waiting time:** total time spent in Ready queue.
- **Response time:** submission $\rightarrow$ first response (interactive).
- **Fairness** and meeting **deadlines** (real-time).

No single scheduler optimizes all criteria; trade-offs are fundamental.

Definition 3.1 (Burst). The *CPU burst* is the interval a process runs before blocking. Bursts are typically exponentially distributed; short bursts dominate, motivating SJF-like policies.

3.2 Classical Algorithms

Consider jobs P_1 – P_3 with bursts $(24, 3, 3)$ arriving at $t = 0$.

3.2.1 FCFS (First-Come, First-Served)

Nonpreemptive, simple FIFO. Suffers *convoy effect*: a long job blocks short ones. Waiting times above: $W = (0, 24, 27)$, avg 17.

3.2.2 SJF / SRTF

SJF (nonpreemptive) picks the shortest next burst — optimal for minimizing average waiting time when all jobs arrive together. With bursts above, order P_2, P_3, P_1 gives $W = (0, 3, 6) \rightarrow$ avg 3 (much better).

SRTF (preemptive SJF / Shortest Remaining Time First) preempts if a new arrival has shorter remaining time.

3.2.3 Priority Scheduling

Each process has a priority; highest runs first. Can cause *starvation* (low-priority jobs never run). Fix: *aging* — gradually increase priority of waiting jobs.

3.2.4 Round Robin (RR)

Preemptive FCFS with time quantum q . Each Ready process gets $\leq q$ before being requeued. Small $q \rightarrow$ good response, high context-switch overhead; large $q \rightarrow$ FCFS. Typically $q = 10$ – 100 ms with context switch $\sim 10\mu$ s.

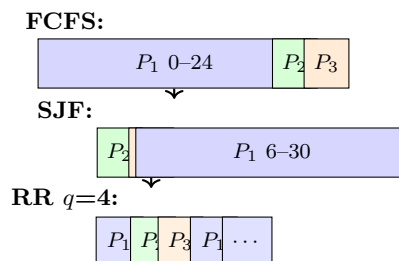


Figure 3.1: Gantt charts for FCFS, SJF, and RR ($q = 4$, schematic) on the same workload.

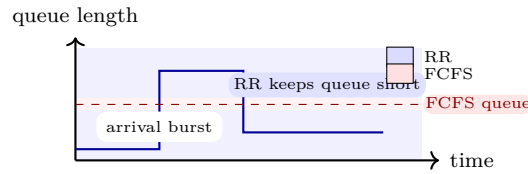


Figure 3.2: Intuition: RR interleaving keeps average queue occupancy lower than FCFS under mixed bursts.

3.2.5 Choosing the Quantum

Too small a q causes thrashing on context switches; too large starves interactive jobs. A rule of thumb: q should be large relative to context-switch cost (e.g., $100\times$) but small relative to human perception (~ 100 ms). Adaptive quanta (as in MLFQ) get the best of both worlds.

3.3 Multilevel Feedback Queue (MLFQ)

MLFQ approximates SJF *without knowing burst lengths*: it uses history. K queues with decreasing priority and increasing quantum.

- New jobs enter Q_0 (highest priority, small q).
- If a job uses its full quantum, it is demoted to Q_{i+1} .
- If it blocks early (I/O-bound), it stays (or is promoted).
- Periodically boost all jobs to Q_0 to avoid starvation.

Result: interactive/I/O-bound jobs stay high; CPU-bound jobs sink, getting longer quanta but less frequent service — mimicking SJF.

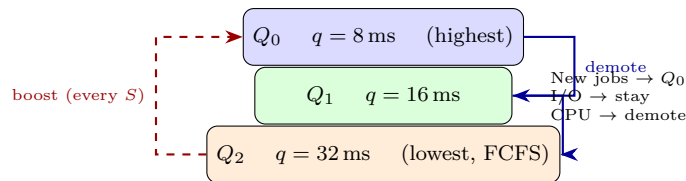


Figure 3.3: Three-level MLFQ: quanta double per level; periodic priority boost prevents starvation.

3.3.1 Multicore and Real-Time

Multicore adds affinity, load balancing, and per-core run queues. Real-time schedulers (Rate-Monotonic, EDF) provide deadline guarantees; Linux CFS uses virtual runtime and a red-black tree for $O(\log n)$ fair scheduling.

Remark 3.1 (Little's Law in Scheduling). For a stable queue, $L = \lambda W$ (average occupancy = arrival rate $\times$ average waiting time). Reducing W via better scheduling directly reduces average queue length — a key intuition behind SJF/MLFQ gains.

```
1 // bursts[] sorted by arrival; compute waiting times
2 long wait_fcfs(int n, int b[]){ long t=0, tot=0;
3     for(int i=0;i<n;i++){ tot+=t; t+=b[i]; } return tot;
4 }
```

Summary: FCFS is simple but convoy-prone; SJF is optimal for avg. wait but needs burst estimates; RR is fair/interactive; MLFQ adapts without estimates.

3.4 Exercises

Exercise 3.1. For bursts $P_1=8, P_2=4, P_3=2$ arriving at 0, compute average waiting time under FCFS (P_1, P_2, P_3), SJF, and RR ($q = 2$). Show Gantt charts.

Exercise 3.2. Prove SJF minimizes average waiting time when all jobs arrive simultaneously. Where does the proof break with different arrival times?

Exercise 3.3. A system has $q = 10$ ms and context switch 0.5 ms. What fraction of CPU is wasted if every burst is 10 ms? How does halving q change it?

Exercise 3.4. Design MLFQ parameters (K , quanta, boost interval S) for a mix of interactive editors and CPU-bound compiles. Justify each choice.

Exercise 3.5. Explain starvation in priority scheduling and how aging fixes it. Give a concrete aging rule.

Chapter 4

Synchronization

Concurrency without synchronization is chaos. This chapter builds the primitives that make correct concurrent programs possible.

Learning Outcomes. After this chapter you will be able to:

- (L1) identify race conditions and define the critical-section problem;
- (L2) explain Peterson’s algorithm and hardware atomics (TSL, CAS);
- (L3) use semaphores and mutexes correctly;
- (L4) use monitors / condition variables and solve producer–consumer;
- (L5) avoid common pitfalls (deadlock, missed wakeup, busy waiting).

4.1 Races and the Critical Section

A *race condition* occurs when unsynchronized accesses to shared state produce nondeterministic results. Example: `count++` is `load-inc-store`; interleaving two increments can lose an update.

4.1.1 Critical-Section Requirements

A correct mutual-exclusion protocol guarantees:

- (i) **Mutual exclusion:** at most one process in its critical section (CS).
- (ii) **Progress:** a process not in CS cannot block others indefinitely.
- (iii) **Bounded waiting:** no indefinite starvation.

4.1.2 Peterson's Algorithm (2 processes)

For processes P_0, P_1 with shared `flag[2]` and `turn`:

Listing 4.1: Peterson's algorithm for P_i

```

1 // shared: flag[2]={0,0}; turn=0;
2 flag[i] = 1; turn = 1 - i;          // intend + yield
3 while (flag[1-i] && turn == 1-i) ;  // busy-wait
4 // --- critical section ---
5 flag[i] = 0;
6 // --- remainder ---

```

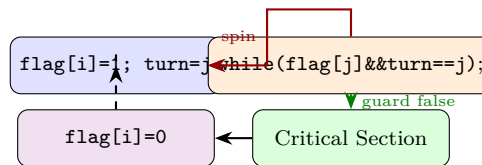


Figure 4.1: Peterson's protocol flow for process P_i (busy-waiting).

Peterson satisfies all three requirements but uses busy waiting, is limited to two processes, and assumes sequentially consistent memory (fails under modern reordering without fences).

4.1.3 Hardware Support

Atomic instructions let software build synchronization without pure busy-wait loops: **test-and-set** (TSL), **compare-and-swap** (CAS), and **fetch-and-add** — each completes atomically in hardware.

Listing 4.2: Spinlock via compare-and-swap

```

1 #include <stdatomic.h>
2 atomic_flag lock = ATOMIC_FLAG_INIT;
3 void acquire() { while (atomic_flag_test_and_set(&lock)) ; /* spin */ }
4 void release() { atomic_flag_clear(&lock); }

```

4.2 Semaphores and Mutexes

A *semaphore* S is an integer with atomic `wait(S) / P` and `signal(S) / V`:

`wait(S): while(S ≤ 0); S-` `signal(S): S++`

Binary semaphore ($S \in \{0,1\}$) acts as a mutex; counting semaphore tracks available resources. Practical semaphores block instead of spinning (queue + `sleep/wakeup`).

4.2.1 Producer–Consumer with Semaphores

Buffer size N : `empty = N`, `full = 0`, `mutex = 1`.

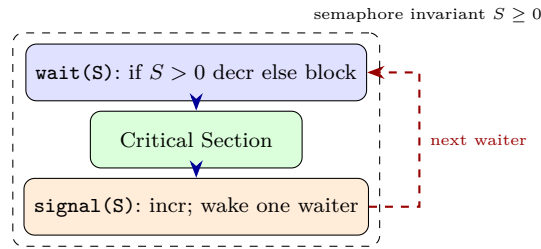


Figure 4.2: Semaphore-based mutual exclusion with blocking (not spinning).

Listing 4.3: Bounded buffer — semaphore solution

```

1 sem_t empty, full, mutex; // init N, 0, 1
2 void *producer(void *a) {
3     for (;;) { Item x = produce();
4         sem_wait(&empty); sem_wait(&mutex);
5         buffer[in] = x; in = (in+1)%N;
6         sem_post(&mutex); sem_post(&full);
7     }
8 }
9 void *consumer(void *a) {
10    for (;;) {
11        sem_wait(&full); sem_wait(&mutex);
12        Item x = buffer[out]; out=(out+1)%N;
13        sem_post(&mutex); sem_post(&empty);
14        consume(x);
15    }
16 }

```

Ordering matters: waiting on mutex before empty/full can deadlock (see Chapter 5).

4.3 Monitors and Condition Variables

A *monitor* encapsulates shared data + procedures with implicit mutual exclusion (only one thread inside at a time). *Condition variables* (wait, signal/broadcast) let threads sleep until a predicate holds — always re-check in a **while** loop (spurious wakeups).

Listing 4.4: Monitor-style bounded buffer (threads)

```

1 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
2 pthread_cond_t not_full = PTHREAD_COND_INITIALIZER,
3     not_empty = PTHREAD_COND_INITIALIZER;
4 int cnt=0, in=0, out=0; Item buf[N];
5 void put(Item x){ pthread_mutex_lock(&m);
6     while(cnt==N) pthread_cond_wait(&not_full,&m);
7     buf[in]=x; in=(in+1)%N; cnt++;
8     pthread_cond_signal(&not_empty); pthread_mutex_unlock(&m);
9 }
10 Item get(){ pthread_mutex_lock(&m);
11     while(cnt==0) pthread_cond_wait(&not_empty,&m);
12     Item x=buf[out]; out=(out+1)%N; cnt--;
13     pthread_cond_signal(&not_full); pthread_mutex_unlock(&m); return x;

```

14 }

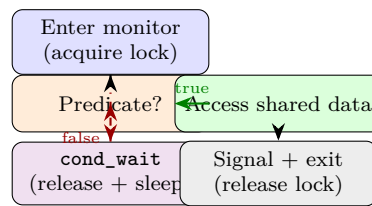


Figure 4.3: Monitor + condition variable pattern: always re-check predicate after wakeup.

Remark 4.1 (Spurious wakeups). On some systems `pthread_cond_wait` may return without a signal. Hence the predicate must be rechecked in a loop — an `if` would proceed with a false precondition.

4.3.1 The Readers–Writers Problem

Many threads may *read* shared data at once, but a *writer* needs exclusive access. Intuition: readers share a library; a writer needs the whole room. The first-readers solution guards a `readcount` with a `mutex`: the first reader locks `wrt` (blocking writers), the last reader unlocks it, and writers simply lock `wrt`. Readers never block each other, so concurrent reads stay fast. The flaw: a steady stream of readers can starve a waiting writer forever—the reader–writer exercise below asks you to flip the priority (at the cost of possible reader starvation). Real systems (Linux `rwsem`, databases) add fairness queues instead.

4.4 Exercises

Exercise 4.1. Show the lost-update race for `count++` with a concrete interleaving of `load/inc/store` instructions.

Exercise 4.2. Prove Peterson’s algorithm guarantees mutual exclusion. Which requirement fails if `turn` is removed?

Exercise 4.3. Swap the two `sem_wait` lines in producer (`mutex` before `empty`). Construct a deadlock scenario.

Exercise 4.4. Implement a reader–writer lock with semaphores that gives writers priority. Explain starvation of readers.

Exercise 4.5. Why must `pthread_cond_wait` be called inside a `while` loop testing the predicate, not an `if`?

Chapter 5

Deadlocks

“A deadlock is a standstill.” Four conditions conspire to freeze a system; this chapter shows how to prevent, avoid, or recover from them.

Learning Outcomes. After this chapter you will be able to:

- (L1) state the four Coffman conditions for deadlock;
- (L2) model resource allocation with graphs and detect cycles;
- (L3) apply deadlock prevention by breaking each condition;
- (L4) run the Banker’s algorithm for deadlock avoidance;
- (L5) compare detection-and-recovery vs. ignoring deadlocks (ostrich).

5.1 System Model and Four Conditions

Resources have types with multiple instances (e.g., 3 tape drives). A process uses a resource as: **request** $\rightarrow$ **use** $\rightarrow$ **release**. A *deadlock* is a set of processes each waiting for a resource held by another in the set, with no progress possible.

Theorem 5.1 (Coffman conditions). Deadlock can occur only if all four hold simultaneously:

- (1) **Mutual exclusion:** at least one resource is non-shareable.
- (2) **Hold and wait:** a process holds one resource while waiting for another.
- (3) **No preemption:** resources cannot be forcibly taken away.
- (4) **Circular wait:** a cycle $P_0 \rightarrow R_0 \rightarrow P_1 \rightarrow R_1 \rightarrow \cdots \rightarrow P_0$.

If any condition is absent, deadlock is impossible. Prevention attacks one of the four; avoidance uses extra information to stay out of danger.

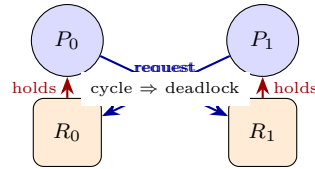


Figure 5.1: Resource-allocation graph with a cycle: mutual exclusion + hold-and-wait + circular wait.

5.2 Resource-Allocation Graphs

A *resource-allocation graph* (RAG) has process nodes P_i , resource nodes R_j , request edges $P_i \rightarrow R_j$ and assignment edges $R_j \rightarrow P_i$.

- No cycle $\Rightarrow$ no deadlock.
- Cycle $\Rightarrow$ deadlock *if* each resource has one instance; otherwise cycle is necessary but not sufficient.

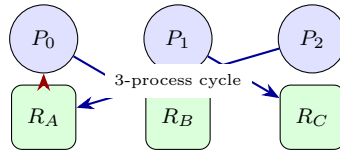


Figure 5.2: A 3-process, 3-resource wait cycle; with single-instance resources this is a deadlock.

Deadlock detection for single-instance resources reduces to cycle detection in the wait-for graph. For multiple instances, a worklist algorithm similar to Banker's is used.

5.3 Prevention and Avoidance

Prevention breaks one Coffman condition:

- Break mutual exclusion: make resources shareable (not always possible).
- Break hold-and-wait: require all resources at once (**request-all**).
- Break no-preemption: allow preemption of held resources.
- Break circular wait: impose a total order on resources; processes request in increasing order only — no cycle can form.

Avoidance requires advance knowledge: each process declares its maximum claim **Max** per resource. The OS grants requests only if the system remains in a *safe state* (some execution order can satisfy all maxima).

5.3.1 Banker's Algorithm

State: **Available**, **Max**, **Allocation**, **Need** = **Max** - **Allocation**. On request $\leq \text{Need} \leq \text{Available}$, tentatively allocate and run the *safety check*: find a sequence $\langle P_{i_1}, \dots, P_{i_n} \rangle$ where each P_{i_k} can finish with **Available** + allocations of earlier processes.

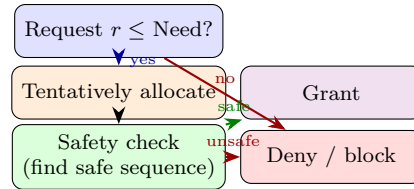


Figure 5.3: Banker’s algorithm: grant only if the tentative state is safe.

Example 5.1 (Banker’s safety check). 5 processes, 3 resource types; **Available** = (3, 3, 2). With Need and Allocation matrices the safe sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ exists, so the state is safe and a new request from P_1 can be granted. If no sequence exists the request must wait, even though resources are free.

Detection-and-recovery lets deadlocks happen, then kills or rolls back processes to break cycles. The “ostrich algorithm” (ignore deadlocks) is used when deadlocks are rare and recovery is expensive — as in many UNIX systems.

Remark 5.1 (Livelock vs. Deadlock vs. Starvation). In deadlock, processes block forever waiting. In livelock, they *actively* retry but keep colliding (e.g., two people stepping aside the same way). Starvation is indefinite postponement of one process while others progress.

5.3.2 Dining Philosophers

Five philosophers share five chopsticks (one between each pair). Each needs two to eat. Naively picking left then right deadlocks when all pick left simultaneously. Solutions: impose ordering (pick lower-numbered first), limit concurrency to four philosophers, or use a waiter (monitor) that grants both sticks atomically.

Listing 5.1: Ordered chopstick acquisition prevents deadlock

```

1  #define N 5
2  pthread_mutex_t chop[N];
3  void eat(int i){
4      int a = i, b = (i+1)%N;
5      if(a > b){ int t=a; a=b; b=t; } // enforce order
6      pthread_mutex_lock(&chop[a]);
7      pthread_mutex_lock(&chop[b]);
8      // ... eat ...
9      pthread_mutex_unlock(&chop[b]);
10     pthread_mutex_unlock(&chop[a]);
11 }

```

The choice among strategies depends on workload: databases use deadlock detection (wait-for graph); embedded real-time systems use prevention (ordering); general-purpose OSes often use the ostrich approach.

5.4 Exercises

Exercise 5.1. Give a real-world analogy for each Coffman condition and explain how breaking that condition resolves the impasse.

Exercise 5.2. Draw the RAG for: P_0 holds R_0 waits for R_1 ; P_1 holds R_1 waits for R_2 ; P_2 holds R_2 waits for R_0 . Is there deadlock if each R_i has two instances and one is free?

Exercise 5.3. Run Banker's algorithm on the textbook 5-process example and verify the safe sequence step by step.

Exercise 5.4. Explain why imposing a total order on resources prevents circular wait. Why is this prevention strategy sometimes impractical?

Exercise 5.5. Compare prevention, avoidance, detection/recovery, and the ostrich approach on overhead and applicability. When would an OS choose each?

Chapter 6

Memory Management

Memory is the workspace of computation. How the OS partitions, translates, and protects it determines both performance and isolation.

Learning Outcomes. After this chapter you will be able to:

- (L1) explain address binding, logical vs. physical addresses;
- (L2) evaluate contiguous allocation, fragmentation, and compaction;
- (L3) describe paging, TLBs, and multi-level page tables;
- (L4) compare paging vs. segmentation and segmented paging;
- (L5) reason about protection, sharing, and page-table overhead.

6.1 Address Binding and Spaces

A program's addresses are *logical* (virtual); hardware executes on *physical* addresses after translation via the MMU. Binding can occur at compile time, load time, or execution time (the most flexible, requiring base + limit or paging hardware).

Logical addresses from the CPU go through relocation (add base) or page-table lookup before reaching RAM. The user never sees physical addresses.

6.1.1 Contiguous Allocation

Early systems placed each process in one contiguous block $[base, base + limit)$. Hardware checks every access: $base \leq addr < base + limit$. Simple and fast, but causes *external fragmentation* — free memory is scattered in holes too small for new processes. *Compaction* (moving processes) is expensive.

Allocation strategies for holes:

- **First-fit:** use first hole large enough — fast, low overhead.

- **Best-fit:** smallest sufficient hole — reduces waste but leaves tiny slivers.
- **Worst-fit:** largest hole — tends to leave usable fragments.

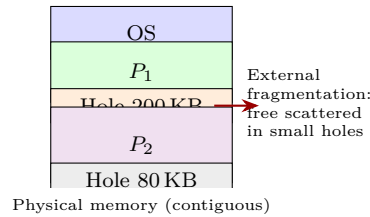


Figure 6.1: Contiguous allocation with external fragmentation.

6.2 Paging

Paging eliminates external fragmentation by dividing logical memory into fixed-size *pages* (e.g., 4 KB) and physical memory into *frames* of the same size. A *page table* maps page number p to frame f ; offset d is preserved: physical address = $f \times \text{PAGE_SIZE} + d$.

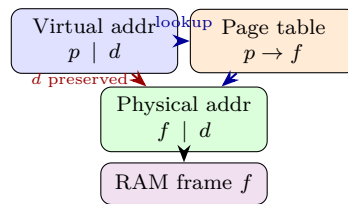


Figure 6.2: Paging address translation: page number maps to frame, offset passes through.

6.2.1 TLB

Every memory access would need a page-table lookup without caching. The *Translation Lookaside Buffer* (TLB) is a small associative cache of recent $p \rightarrow f$ mappings. On a TLB miss, hardware or OS walks the page table and refills the TLB. Effective access time:

$$t_{\text{eff}} = h \cdot (t_{\text{TLB}} + t_{\text{mem}}) + (1 - h) \cdot (t_{\text{TLB}} + 2t_{\text{mem}})$$

with hit rate h .

Example 6.1 (TLB effect). With $h = 0.98$, $t_{\text{TLB}} = 5 \text{ ns}$, $t_{\text{mem}} = 80 \text{ ns}$: $t_{\text{eff}} = 0.98 \cdot 85 + 0.02 \cdot 165 = 86.6 \text{ ns}$ vs. 160 ns without TLB. Even a 2% miss rate doubles cost without the TLB.

6.2.2 Multi-Level Page Tables

A flat page table for 32-bit addresses with 4 KB pages has 2^{20} entries ($\sim 4 \text{ MB}$ per process). *Two-level* paging splits p into $p_1 | p_2$: the outer table points to inner tables allocated only when needed, drastically reducing overhead for sparse address spaces. 64-bit systems use 3–4 levels.

6.3 Segmentation and Combined Schemes

Segmentation divides memory logically (code, data, stack, heap) into variable-size segments with base/limit per segment. It matches programmer view and enables sharing/protection per segment, but reintroduces external fragmentation.

Segmented paging: each segment has its own page table — combining logical organization with paging’s freedom from external fragmentation. Intel x86 historically used this hybrid.

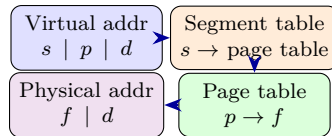


Figure 6.3: Segmented paging: segment $\rightarrow$ page table $\rightarrow$ frame.

Paging suffers *internal fragmentation* (last page partially unused, average $\frac{1}{2}$ page per segment). Protection bits (R/W/X, user/supervisor) and sharing (shared frames, copy-on-write) are enforced per page/segment.

Remark 6.1 (Internal vs. external fragmentation). Paging has no external fragmentation (any free frame fits any page) but pays internal fragmentation: the last page of each process wastes on average $\text{PAGE_SIZE}/2$ bytes. With 4KB pages and many small processes this adds up, motivating variable page sizes (huge pages: 2MB/1GB) for large mappings.

Listing 6.1: Page-table walk (two-level, sketch)

```

1 // va = p1 | p2 | offset (10 | 10 | 12 for 4KB pages, 32-bit)
2 pte_t *pt = outer[p1];           // outer table lookup
3 if (!pt) fault("outer miss");
4 frame = pt[p2].frame;           // inner table lookup
5 phys = (frame << 12) | offset;
  
```

Remark 6.2 (Why TLB shutdown matters). On multicore, updating a page-table entry requires invalidating stale TLB entries on other cores (TLB shutdown via IPIs). Frequent `mmap/mprotect` can thus be surprisingly expensive on large machines.

Example 6.2 (Two-level page-table size). For a 32-bit VA with 4KB pages, p_1 and p_2 are 10 bits each. The outer table has 1024 entries; each inner table has 1024 entries. A process using only two far-apart pages needs just 1 outer entry + 2 inner tables (~ 12 KB) instead of a flat 4 MB table — a $300\times$ saving.

Remark 6.3 (Huge pages). Using 2 MB or 1 GB huge pages reduces TLB pressure (one entry covers more) at the cost of higher internal fragmentation. Databases and HPC workloads often enable huge pages for this reason.

6.4 Exercises

Exercise 6.1. Given logical address 0x1234 with page size 1 KB (10-bit offset), find page number and offset. If page 4 maps to frame 7, what is the physical address?

Exercise 6.2. For $t_{\text{mem}} = 100\text{ ns}$, $t_{\text{TLB}} = 10\text{ ns}$, and hit rate 95%, compute effective access time with and without TLB.

Exercise 6.3. Compare best-fit vs. first-fit allocation on a sequence of allocations and frees. Which fragments more and why?

Exercise 6.4. Why do 64-bit systems need multi-level page tables? Estimate table size for a flat table with 4 KB pages.

Exercise 6.5. Explain how copy-on-write for `fork` uses page-table protection bits to defer copying until a write occurs.

Chapter 7

Virtual Memory

Virtual memory lets a process believe it owns a large, contiguous address space backed by disk. This chapter explains demand paging, replacement, and thrashing.

Learning Outcomes. After this chapter you will be able to:

- (L1) explain demand paging, page faults, and effective access time;
- (L2) simulate FIFO, Optimal, LRU, and Clock page replacement;
- (L3) define working set, thrashing, and frame allocation;
- (L4) reason about copy-on-write, memory-mapped files, and swapping;
- (L5) evaluate VM performance via fault rate and locality.

7.1 Demand Paging

In *demand paging*, pages are loaded only when accessed. A page table entry has a *valid* bit (in memory), *dirty* bit, and reference bit. On access to an invalid page, a *page fault* traps to the kernel, which finds a free frame (or evicts one), reads the page from disk, updates the table, and restarts the faulting instruction.

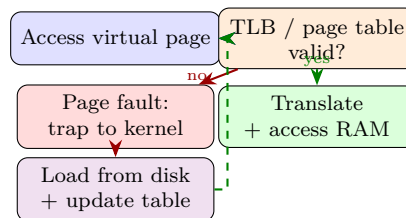


Figure 7.1: Demand paging flow: fast path vs. page-fault path.

Effective access time with fault rate p and fault service time t_f :

$$t_{\text{eff}} = (1 - p) t_{\text{mem}} + p t_f, \quad t_f \approx 8\text{--}20 \text{ ms (disk I/O)}.$$

Even $p = 0.001$ with $t_f = 10 \text{ ms}$ and $t_{\text{mem}} = 100 \text{ ns}$ gives $t_{\text{eff}} \approx 10 \mu\text{s}$ — $100\times$ slower. Locality and good

replacement keep p tiny.

Example 7.1 (Effective access time). If $p = 10^{-4}$, $t_{\text{mem}} = 100 \text{ ns}$, $t_f = 8 \text{ ms}$: $t_{\text{eff}} = 0.9999 \cdot 100 \text{ ns} + 10^{-4} \cdot 8 \text{ ms} \approx 900 \text{ ns}$ — almost $10\times$ slower than no faults. Keeping $p < 10^{-5}$ is essential.

7.1.1 Copy-on-Write and Memory-Mapped Files

`fork` shares pages read-only; a write fault copies the page. `mmap` maps files directly into the address space, with demand paging handling I/O — the basis for shared libraries and fast file access.

7.2 Page Replacement

When no free frame exists, the OS must evict a page. Goals: minimize future faults, avoid Belady’s anomaly (more frames $\rightarrow$ more faults, as in FIFO).

- **FIFO**: evict oldest page — simple, suffers Belady’s anomaly.
- **Optimal (OPT)**: evict page whose next use is farthest — optimal but requires future knowledge; used as benchmark.
- **LRU**: evict least recently used — exploits temporal locality; implement via counters, stack, or reference bits.
- **Clock (second-chance)**: circular list with reference bits; on fault, sweep: clear $R = 1$ and advance, evict first $R = 0$ page.

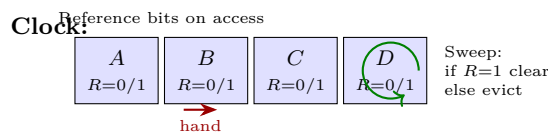


Figure 7.2: Clock (second-chance) replacement: circular sweep over reference bits.

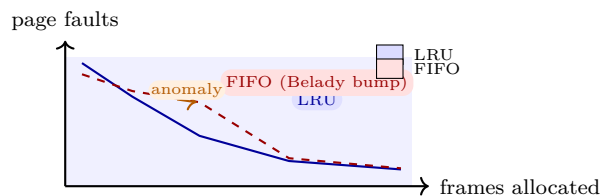


Figure 7.3: Page faults vs. frames: LRU monotonic; FIFO can exhibit Belady’s anomaly.

Remark 7.1 (Belady’s anomaly intuition). FIFO evicts by age, not recency. A longer queue can retain “old but hot” pages while evicting recently used ones that FIFO happened to admit later. Stack algorithms (LRU, OPT) never suffer this because their resident sets are nested: $M(k) \subseteq M(k+1)$.

Example 7.2 (Reference string: 1,2,3,4,1,2,5 – 3 frames). FIFO faults on 1,2,3,4,5 (7 faults with Belady cases); LRU faults on 1,2,3,4,5 (similar here but better on locality-heavy strings); Clock approximates LRU with one reference bit per page.

7.3 Thrashing and Frame Allocation

Thrashing occurs when the system spends more time paging than executing, typically when total working sets exceed available frames. The *working set* $W(t, \Delta)$ is the set of pages referenced in the last Δ accesses; allocating fewer frames than $|W|$ guarantees high fault rates.

Cures: working-set model (ensure each process keeps its W resident), page-fault frequency control, and simply admitting fewer processes (*load control*). Global replacement can let one process thrash others; local (per-process) replacement isolates faults.

Listing 7.1: Simulating LRU faults

```

1  #include <stdio.h>
2  #define FRAMES 3
3  int lru_faults(int *ref, int n){
4      int frame[FRAMES], last[FRAMES], t=0, faults=0;
5      for(int i=0; i<FRAMES; i++) frame[i]=-1, last[i]=-1;
6      for(int k=0; k<n; k++){ int p=ref[k], hit=0;
7          for(int i=0; i<FRAMES; i++) if(frame[i]==p){ hit=1; last[i]=t++; break; }
8          if(!hit){ faults++; int v=0; // evict LRU
9              for(int i=1; i<FRAMES; i++) if(last[i]<last[v]) v=i;
10             frame[v]=p; last[v]=t++;
11         }
12     } return faults;
13 }
```

7.3.1 Frame Allocation and Locality

Allocate too few frames and the process faults constantly; too many and CPU utilization drops (fewer processes fit). The working-set window Δ should span a locality phase; sampling reference bits periodically estimates it. Thrashing is visible as high paging I/O with low CPU use — the system pages furiously but makes no progress.

Remark 7.2 (Prepaging). *Prepaging* (prefetching pages before they fault) helps when locality is predictable (sequential file access). Mispredicted prepaging wastes I/O and frames, so most systems use demand paging with limited speculative read-ahead.

7.4 Exercises

Exercise 7.1. Given $t_{\text{mem}} = 100 \text{ ns}$, $t_f = 10 \text{ ms}$, what fault rate keeps $t_{\text{eff}} < 200 \text{ ns}$?

Exercise 7.2. Simulate FIFO, LRU, and Clock on reference string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 with 3 and 4 frames. Identify any Belady anomaly.

Exercise 7.3. Explain why LRU is a stack algorithm (more frames cannot increase faults) while FIFO is not. Give a concrete Belady example.

Exercise 7.4. Define working set $W(t, \Delta)$. How does Δ affect thrashing detection? What happens if Δ is too small or too large?

Exercise 7.5. Describe copy-on-write for `fork` in terms of page-table bits and fault handling. When is the copy actually performed?

Chapter 8

File Systems, I/O, and Protection

Files persist beyond processes; I/O connects computation to the world; protection decides who may do what. This chapter unifies storage management with security.

Learning Outcomes. After this chapter you will be able to:

- (L1) compare file allocation methods (contiguous, linked, FAT, indexed/inode);
- (L2) explain directory structures and journaling for crash consistency;
- (L3) describe disk scheduling (FCFS, SSTF, SCAN, C-SCAN, LOOK);
- (L4) outline I/O subsystem organization and buffering;
- (L5) reason about protection (ACLs vs. capabilities) and security threats.

8.1 Files and Allocation

A *file* is a named collection of bytes with attributes (name, size, permissions, timestamps) stored in a directory entry. Allocation decides how blocks are placed on disk:

- **Contiguous:** file occupies consecutive blocks — fast sequential access, but external fragmentation and resizing difficulty.
- **Linked:** blocks linked via pointers — no fragmentation, but random access is $O(n)$ and pointers can be lost.
- **FAT (File Allocation Table):** linked allocation with a central table in memory — supports random access via table walk; used in FAT32/exFAT.
- **Indexed (inode):** each file has an index block (inode) with direct, single-, double-, and triple-indirect pointers — efficient for both small and large files (UNIX, ext4).

Example 8.1 (Inode capacity). With 4 KB blocks and 4-byte pointers, one indirect block holds 1024 pointers.

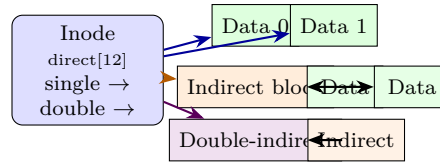


Figure 8.1: UNIX inode: 12 direct pointers + single/double/triple indirect for large files.

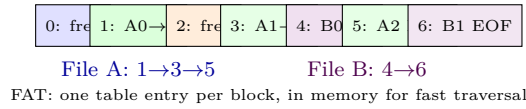


Figure 8.2: FAT allocation: central table maps each block to next block or EOF.

12 direct + single + double indirect covers $12 + 1024 + 1024^2 \approx 10^6$ blocks ≈ 4 GB. Triple-indirect reaches terabytes.

8.1.1 Directories and Journaling

Directories map names $\rightarrow$ inodes (or FAT entries). Modern file systems use *journaling* (write-ahead log) to ensure crash consistency: metadata changes are first written to a journal, then checkpointed. On crash, replay the journal. Alternatives: copy-on-write (ZFS, btrfs) and soft updates.

Free-space management uses bitmaps (1 bit per block) or free lists.

8.2 Disk Scheduling and I/O

Disk I/O time = seek + rotation + transfer; seek dominates for HDDs. Scheduling reorders the request queue:

- **FCFS**: fair, no starvation, poor seek optimization.
- **SSTF**: nearest track first — good throughput, starvation possible.
- **SCAN (elevator)**: sweep arm back and forth, servicing along the way.
- **C-SCAN**: circular sweep (one direction only) for uniform wait.
- **LOOK / C-LOOK**: SCAN/C-SCAN but reverse at last request, not disk end.

For SSDs, seek is negligible; scheduling focuses on wear leveling and parallelism.

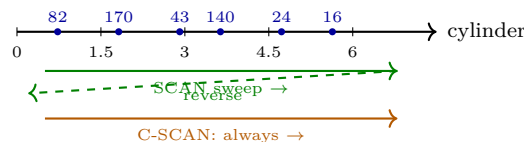


Figure 8.3: Disk scheduling: request cylinders and SCAN vs. C-SCAN sweep direction (schematic).

Example 8.2 (SCAN vs. C-SCAN fairness). SCAN gives edge cylinders shorter waits on the return sweep, biasing toward recently visited tracks. C-SCAN wraps to the far end for near-uniform expected wait, at the cost of one long seek. C-LOOK avoids seeking to the disk end.

8.2.1 I/O Subsystem

I/O is layered: application → kernel I/O subsystem (caching, spooling, buffering) → device drivers → controllers (DMA, interrupts). Polling wastes CPU; interrupt-driven I/O and DMA free the CPU during transfers.

Remark 8.1 (Buffering and caching). The kernel buffers disk blocks in a page cache, coalesces writes, and prefetches sequential reads. Double buffering overlaps computation with I/O: while one buffer is processed, the next is filled by DMA.

8.3 Protection and Security

Protection controls access of subjects (users/processes) to objects (files, devices) via permissions. Two representations:

- **ACL (Access Control List)**: per object, list of (subject, rights).
- **Capability list**: per subject, list of (object, rights).

UNIX uses ACL-like mode bits (owner/group/other: rwx) plus extended ACLs.

Security defends against threats: malware, buffer overflows, privilege escalation, and side channels. Principles: least privilege, defense in depth, and complete mediation. Encryption at rest, authentication, and auditing complement access control.

Listing 8.1: Checking UNIX permissions (sketch)

```

1 #include <sys/stat.h>
2 int can_read(const char *path, uid_t uid, gid_t gid){
3     struct stat st; stat(path, &st);
4     if(uid==st.st_uid) return st.st_mode & S_IRUSR;
5     if(gid==st.st_gid) return st.st_mode & S_IRGRP;
6     return st.st_mode & S_IROTH;
7 }
```

Remark 8.2 (SSD vs. HDD scheduling). On SSDs, seek time is negligible and wear leveling plus internal parallelism dominate. The OS focuses on TRIM/discard, write amplification, and queue depth (NVMe) rather than cylinder ordering.

8.4 Exercises

Exercise 8.1. For a 4 KB block and 32-bit block pointers, how many blocks does a 12-direct + single + double indirect inode address? What file size does that cover?

Exercise 8.2. Given disk queue 98, 183, 37, 122, 14, 124, 65, 67 with head at 53, compute total seek distance for FCFS, SSTF, SCAN, and C-SCAN (disk 0–199, SCAN goes to end).

Exercise 8.3. Explain how journaling guarantees crash consistency. What is replayed after a crash, and what is discarded?

Exercise 8.4. Compare ACLs vs. capabilities for revocation and least-privilege delegation. Give an example where each shines.

Exercise 8.5. Why does SSD scheduling differ from HDD scheduling? What replaces seek optimization as the primary concern?